4) Left Shift : <<

Bitwise Operators in C++ (Part -2)

```
int main()
{   int x = 3;
    cout << (x << 1) << endl;
    cout << (x << 2) << endl;
    int y = 4;
    int z = (x << y);
    cout << z;
    return 0;
}
```

Output: 6
12
48

x : 0000..000011

x<<1 : 0000..000110

x<<2 : 0000..001100

x<<y : 0000..110000

⇒ Remove and Ignore leading y bits

⇒ Move remaing 32-y bits to leftmost

⇒ Abbend y 0s at the end

5) Right Shift : >>

```
int main()
{   int x = 33;
    cout << (x >> 1) << endl;
    cout << (x >> 2) << endl;
    int y = 4;
    int z = (x >> y);
    cout << z;
    return 0;
}
```

Output: 16
8
2

x : 0000..100001 (x x x x)

x>>1 : 0000..010000

x>>2 : 0000..001000

x>>y : 0000..000010

⇒ Remove and Ignore trailing y bits

⇒ Move remaing 32-y bits to rightmost

⇒ Add y 0s at the beginning

(*) $x << y$ is equivalent to $x * 2^y$ if leading y bits are 0 in x.

x	y	$x << y$	
3	1	6	(3×2^1)
3	2	12	(3×2^2)
3	4	48	(3×2^4)

(*) $x >> y$ is equivalent to $\lfloor x/2^y \rfloor$

x	y	$x >> y$	
33	1	16	$\lfloor 33/2^1 \rfloor$
33	2	8	$\lfloor 33/2^2 \rfloor$
33	4	2	$\lfloor 33/2^4 \rfloor$

6) Bitwise Not: ~

x = 1:

00...01

~x

11...10

x = 5:

00...101

~x

11...010

```
int main()
{   unsigned int x = 1;
    cout << (~x) << endl;

    x = 5;
    cout << (~x) << endl;

    return 0;
}
```

Output:

4294967294
4294967290

Please Note: $2^{32} - 1 = 4294967296 - 1$
$= 4294967295$

6) Bitwise Not : ~ (Signed Numbers)

$(-2^{31}$ to $2^{31}-1)$

$x = 1$:

00...01

~x

11...10 $(2^{32}-1-1)$

$x = 5$:

00...101

~x

11...010

$(2^{32}-1-5)$

```
int main()
{ int x = 1;
  cout << (~x) << endl;
  x = 5;
  cout << (~x) << endl;
  return 0;
}
```

Output:

-2

-6

$$\boxed{2^s \text{ Complement of } x = 2^{32} - x}$$